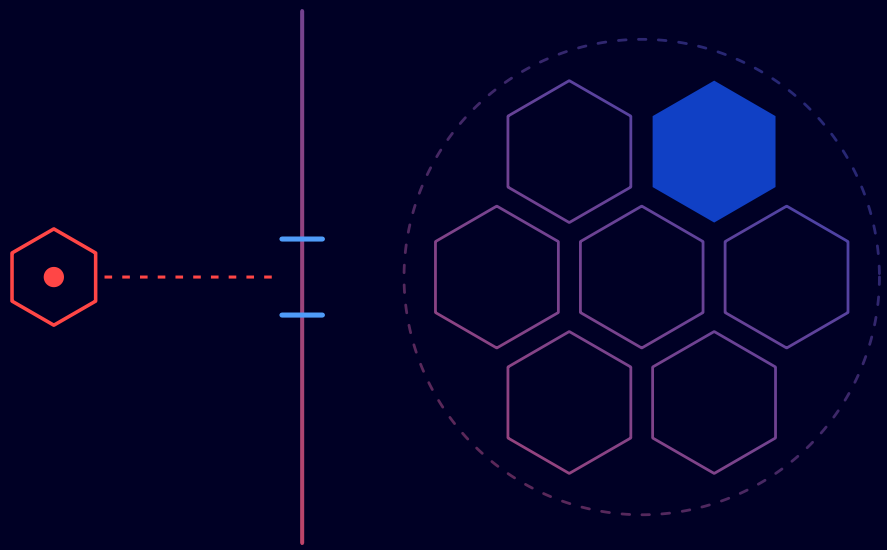
AccuKnox

Fix the cluster before a bad pod gets admitted

Six do's and don'ts for KSPM and admission control, from CIS scans to RBAC, with the AccuKnox setup page for each gap.



The problem

Most teams had a Kubernetes incident last year

Security incidents in containers and Kubernetes are common, and they slow releases down. The cause is often a setting that nobody checked before the workload went live.

89%

of organizations had at least one container or Kubernetes security incident in the last 12 months.

Red Hat, State of Kubernetes Security Report 2024

67%

of respondents delayed or slowed down deployment because of security concerns.

Red Hat, State of Kubernetes Security Report 2024

54%

of GKE clusters are exposed to the internet. The figure is 39% for EKS and 34% for AKS.

Datadog, State of Cloud Security 2025

Root causes

Four habits keep clusters exposed

Each habit is a default or a shortcut. Each one has a control in the six practices that follow.



Defaults favor running

Kubernetes mounts a service account token into every pod by default. A pod with no security context runs as whatever the image says, often root.



Manifests ship unchecked

A password in a Deployment env or a privileged flag passes review because nobody scans the YAML. 40% of organizations detected misconfigurations in their container or Kubernetes environments.



RBAC only grows

Teams add roles and bindings with each release and rarely remove them. Only 2% of granted permissions are in use, so the rest is attack surface.



Nothing guards the door

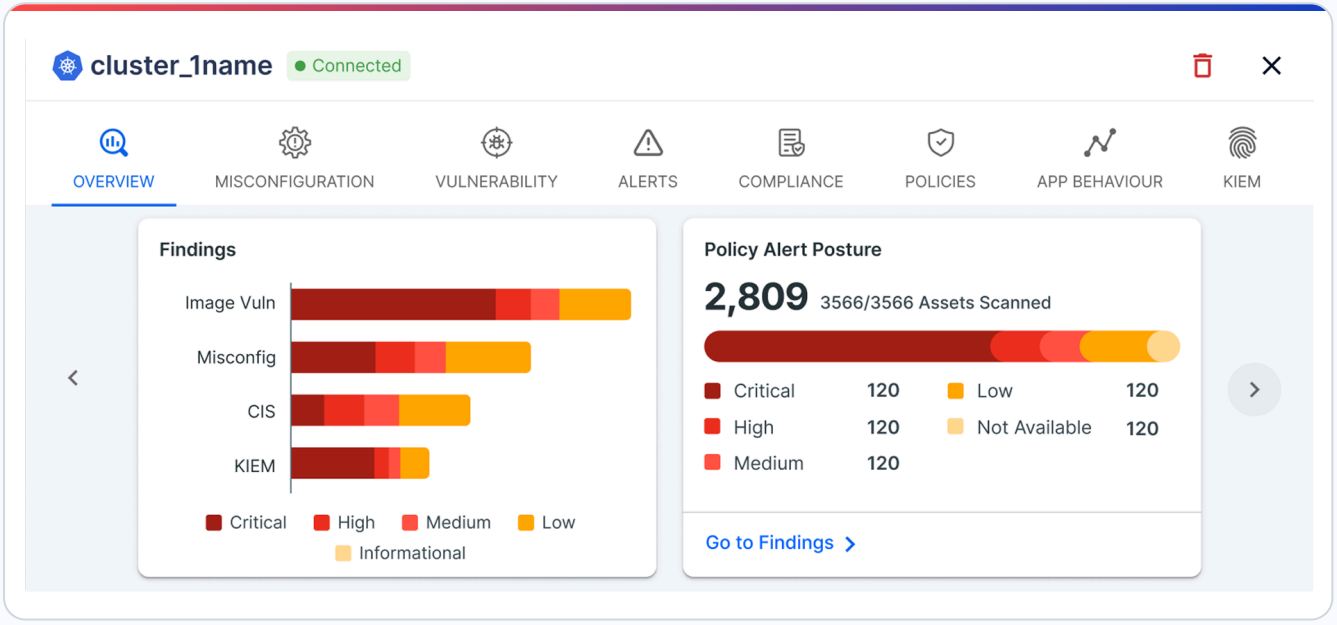
Without admission control, the API server accepts any image from any registry and any pod spec. A scan after deploy reports the risk, but the pod already runs.

Sources. 40%, Red Hat, State of Kubernetes Security Report 2024. 2%, Sysdig, 2024 Cloud Native Security and Usage Report. Token default, help.accuknox.com/use-cases/hardening.

Start here

Six controls cover the cluster posture

Three practices find what is wrong in a running cluster. Three stop the next bad workload or permission at the door. One Helm command onboards the cluster with each job you turn on.



Cluster overview in Inventory, with misconfiguration, CIS and KIEM findings. help.accuknox.com/use-cases/kspm/

01 **CIS benchmarking** 05
Score each cluster against CIS

02 **Misconfiguration scanning** 06
Find the bad line in the manifest

03 **KnoxGuard admission control** 07
Block risky pods at deploy

04 **Pod Security Admission** 08
Set a level per namespace

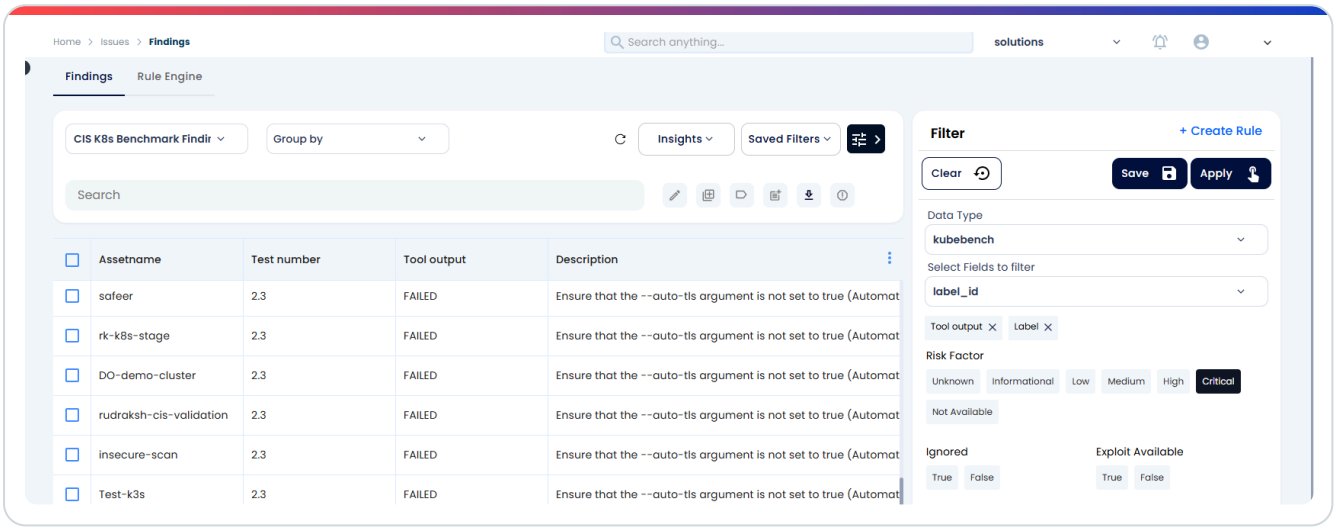
05 **KIEM** 09
Cut unused RBAC permissions

06 **Network segmentation** 10
Allow only observed traffic

Practice 01

Benchmark every cluster against CIS on a schedule

The CIS K8s Benchmarking job runs as a cron job and lists each failed control as a finding.



CIS K8s Benchmark findings, filtered to failed controls. help.accuknox.com/use-cases/cis-benchmarking/

DO

- Turn on Kubernetes CIS Benchmarking for each cluster in **Settings** > **Manage Clusters**.
- Pick the right platform, such as GKE, because the platform decides which controls apply.
- Filter findings to Risk Factor Critical and Tool Output Failed, and fix those first.
- Keep the cron schedule, so drift shows up as a new failed control.

DON'T

- Don't treat one clean scan as compliance. The next change can undo a fix.
- Don't skip control plane checks, such as etcd peer authentication and kubelet certificate checks.
- Don't change a flag before you read the finding's Solution tab.

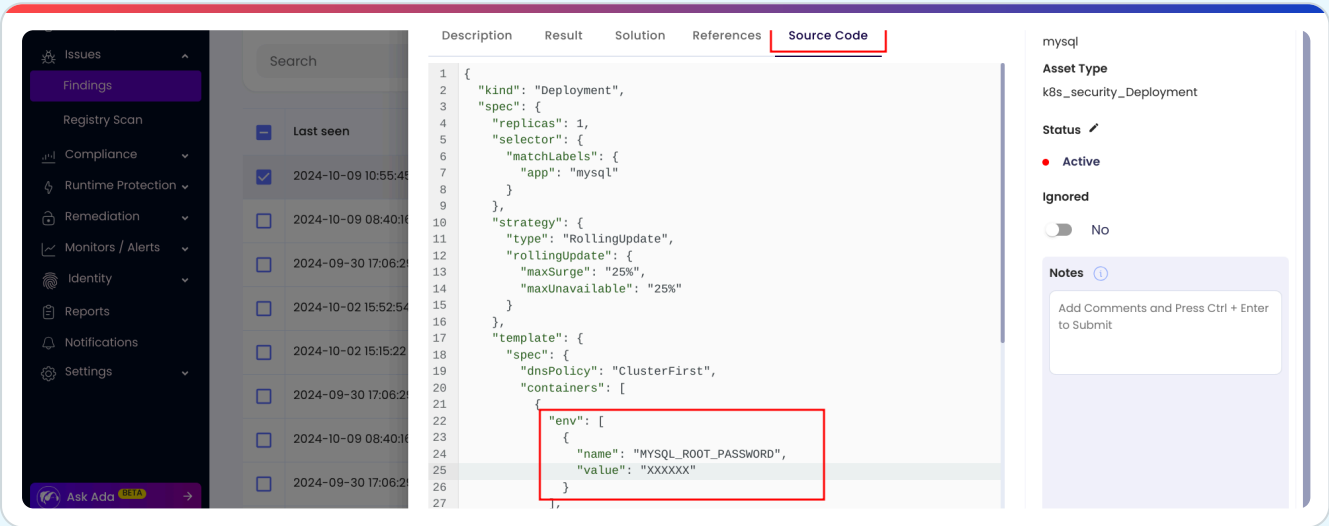
HOW ACCUKNOX CLOSES THE GAP

Each failed control opens with its description, result and solution. Managed cluster benchmarking supports GKE today. Set up CIS benchmarking

Practice 02

Catch the misconfiguration in the manifest

The misconfiguration scan checks for root containers, privilege escalation and more than 100 other rules.



A root database password found in a Deployment env. help.accuknox.com/use-cases/cluster-misconfiguration-scanning/

DO

- Turn on the Cluster Misconfiguration job and set a label for the workloads it scans.
- Select **Cluster Findings** in **Issues** > **Findings** after each scan.
- Open the Source Code tab to see the exact line that failed.
- Create a Jira ticket from the finding, so the workload owner gets the fix.

DON'T

- Don't put credentials in a pod spec or ConfigMap. Use Kubernetes secrets or a key management system.
- Don't run containers as root or allow privilege escalation by default.
- Don't ignore findings for anonymous access. An open endpoint needs no stolen credential.

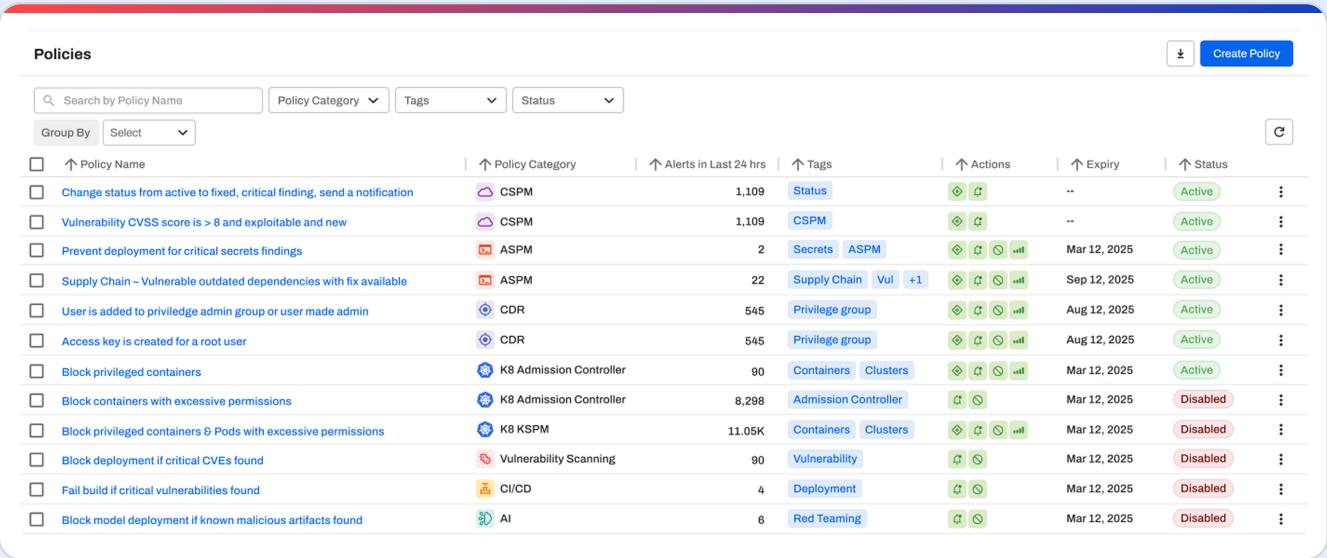
HOW ACCUKNOX CLOSES THE GAP

The scan runs as a Kubernetes cron job. Each finding carries a Solution tab with the failed path to change. Onboard a cluster for misconfiguration scans

Practice 03

Block a risky workload before it runs

KnoxGuard adds admission control through Kyverno, so the API server rejects a pod that breaks policy.



Admission controller policies in the AccuKnox policy list. help.accuknox.com/use-cases/admission-controller-knoxguard/

DO

- Install the AccuKnox agents, then Kyverno, then KnoxGuard with Helm.
- Deny privileged containers per namespace with an AdmissionPolicy set to Block.
- Allow only trusted container registries, at the cluster or namespace level.
- Review blocks in **Monitors** > **Alerts** with the type set to Admission Controller.

DON'T

- Don't let a manifest pull from any registry. An attacker image can open a reverse shell on deploy.
- Don't count on a scan after deploy to stop a privileged pod. The pod already runs.
- Don't keep violation logs in one console. Forward them to your SIEM.

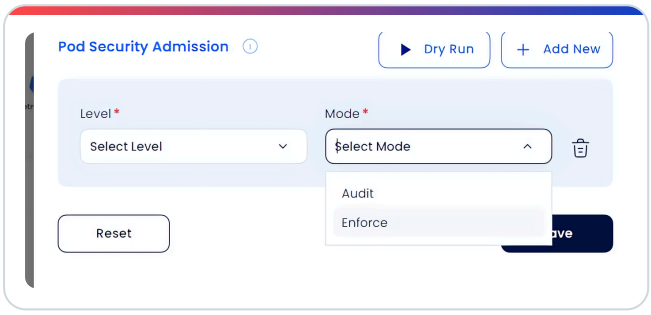
HOW ACCUKNOX CLOSES THE GAP

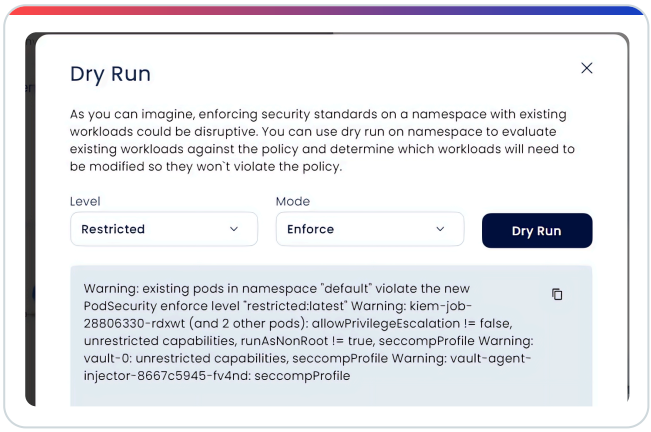
Upload an AdmissionPolicy YAML through Create Policy, and KnoxGuard enforces it through Kyverno. Image vulnerability thresholds at admission are not yet available. Deploy KnoxGuard

Practice 04

Set a Pod Security level on every namespace

Pod Security Admission applies the Privileged, Baseline or Restricted standard per namespace, in Audit or Enforce mode.





Dry run a level before you enforce it. help.accuknox.com/use-cases/pod-security-admission-controller/

DO

- Open **Inventory** > **Clusters**, select View Workloads, then click the cog next to the namespace.
- Start in Audit mode. Violations raise an alert and the pod still runs.
- Run Dry Run before Enforce to list each pod that would break.
- Move application namespaces to Restricted once the dry run is clean.

DON'T

- Don't leave a namespace at Privileged. That level allows known privilege escalations.
- Don't switch to Enforce without a dry run. Enforce rejects every pod that violates the level.
- Don't settle for Baseline on app namespaces. Baseline is the minimally restrictive level.

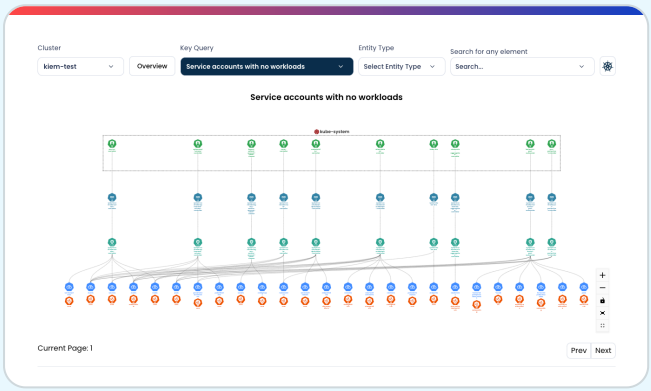
HOW ACCUKNOX CLOSES THE GAP

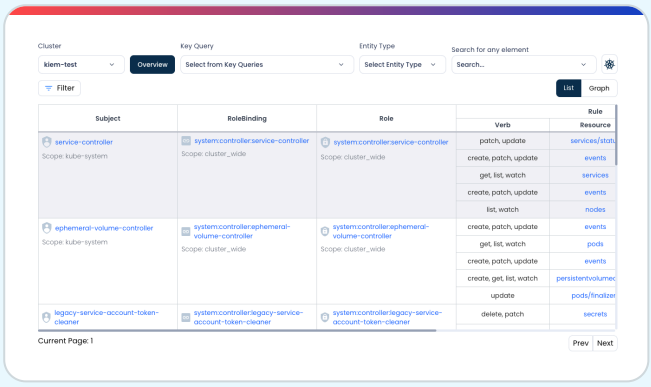
Under Restricted, a pod without runAsNonRoot, a dropped capabilities list and a seccomp profile gets a Forbidden error. Turn on Pod Security Admission

Practice 05

Remove every RBAC permission nobody uses

KIEM maps users, service accounts, roles and bindings in each cluster, then flags risky ones with 15 built in queries.





Service accounts with no workloads, and the permission map behind each binding. help.accuknox.com/use-cases/kiem/

DO

- Install the KIEM job from Manage Cluster and schedule its cron scan.
- Run the query for service accounts with no workloads, and remove the dormant ones.
- List roles with read access to secrets, and trim each to the workloads that need it.
- Open the graph view to trace a binding to every resource it can touch.

DON'T

- Don't keep unused roles. They hold permissions an attacker can pick up.
- Don't grant a cluster wide role where a namespaced role does the job.
- Don't audit RBAC by reading YAML by hand. Subjects, bindings and roles form a graph.

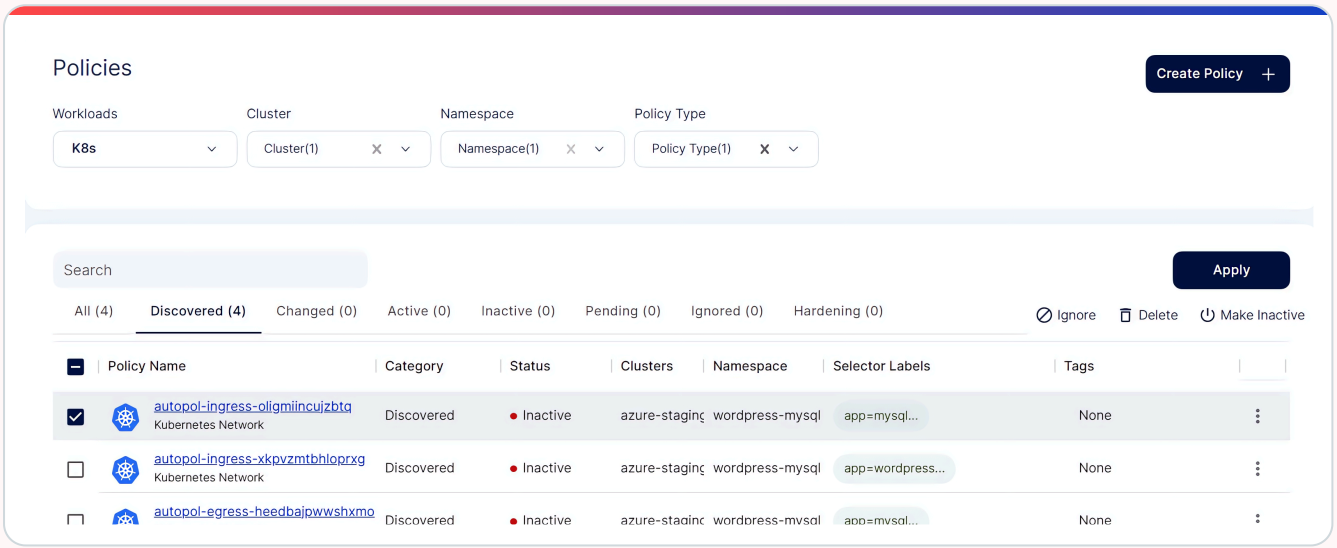
HOW ACCUKNOX CLOSES THE GAP

KIEM queries include principals with excessive privileges and roles that can modify workloads. Find them in **Identity** > **KIEM**. Set up KIEM

Practice 06

Segment workloads so one pod cannot reach all

Kubernetes allows pod to pod traffic by default. AccuKnox discovers each workload's flows and proposes a policy for them.



Discovered ingress and egress policies for one application. help.accuknox.com/use-cases/network-segmentation/

DO

- Review the discovered network policies for each application in the Policies screen.
- Apply ingress and egress policies per workload, one application at a time.
- Approve each applied policy, then check that it moves from Pending to Active.
- Group workloads into segments by application tier or compliance scope.

DON'T

- Don't treat a namespace as a network boundary. Pods in other namespaces can still connect.
- Don't write allow rules from memory. Base them on observed traffic.
- Don't let a database pod accept traffic from every workload in the cluster.

HOW ACCUKNOX CLOSES THE GAP

Policy discovery reads each pod's network connections and builds allow list policies. An applied policy waits in Pending until an approver makes it Active. Segment a workload

Gap check

Find your gap, then turn on the right control

Read each row. Where the statement is true for your clusters, take the first step this week.

IF THIS IS TRUE	ACCUKNOX CAPABILITY	FIRST STEP
Nobody knows which CIS controls fail on each cluster	CIS K8s Benchmarking	Turn on the CIS job in Manage Clusters and set the platform
Manifests reach the cluster with no config scan	Cluster Misconfiguration scanning	Turn on the Cluster Misconfiguration job with a workload label
A privileged pod or unknown registry image deploys with no block	KnoxGuard admission controller	Install Kyverno and KnoxGuard, then upload a deny privileged policy
Namespaces carry no Pod Security level	Pod Security Admission	Set Audit mode, then dry run Restricted per namespace
Nobody can say which service accounts hold unused permissions	KIEM	Install the KIEM job and run the dormant service account query
Any pod can open a connection to any other pod	Network segmentation	Review and apply the discovered policies for one application

AccuKnox

See your cluster posture in one session

Bring this gap check to a demo. An AccuKnox engineer onboards a cluster with you and walks through the first CIS, misconfiguration and KIEM findings.

Book a demo | Open the KSPM docs

KuppingerCole 2026
Featured in the CNAPP Leadership Compass, which cites Kubernetes security and eBPF runtime enforcement.

Two 2026 recognitions
Best Practices awards for securing the AI stack, globally, and for cloud security in Asia Pacific.

Runtime landscape
Named in the Runtime Environment group for kernel level, out of band enforcement.

Source. accuknox.com/analyst-recognition

accuknox.com · © 2026 AccuKnox, Inc.